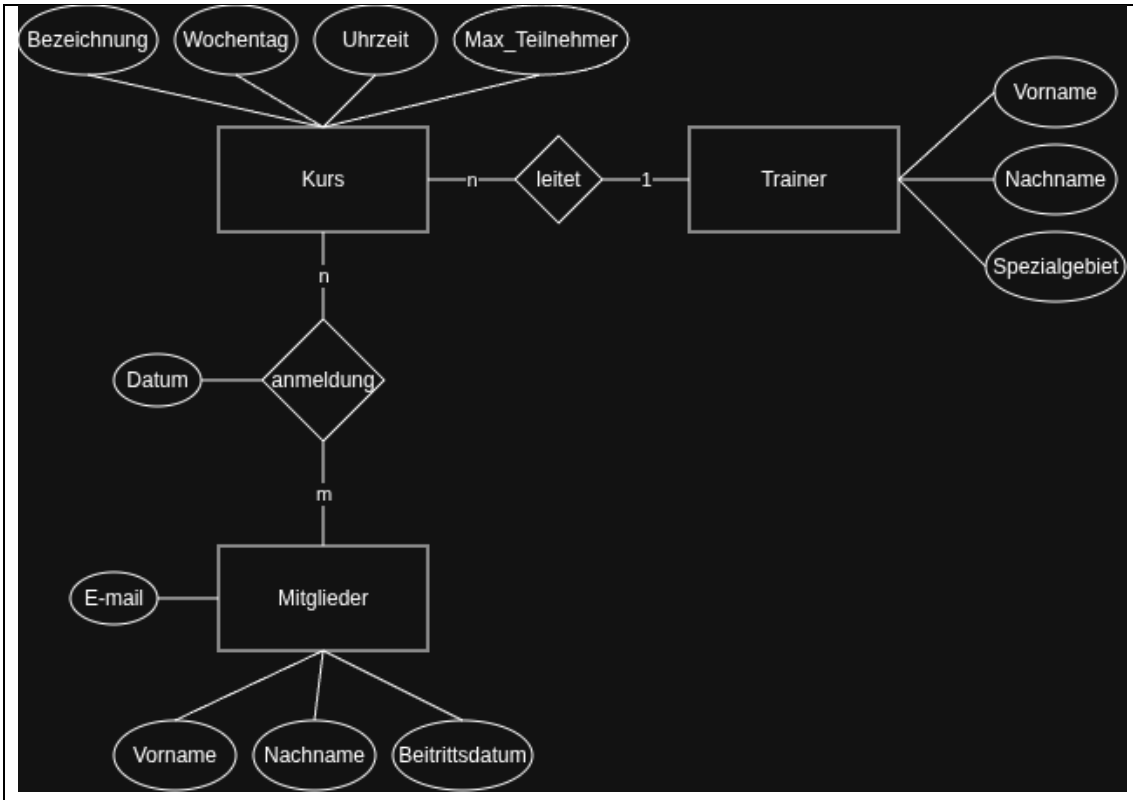
Name: Neicovcen Vladislav **Datum:** 08.04.2026 **Klasse:** 3AHIF

3. Grafische Leistungsfeststellung – DBI1
Thema: Fitnessstudio-Verwaltung (ERM, RM, Python/SQLite3, Anvil)

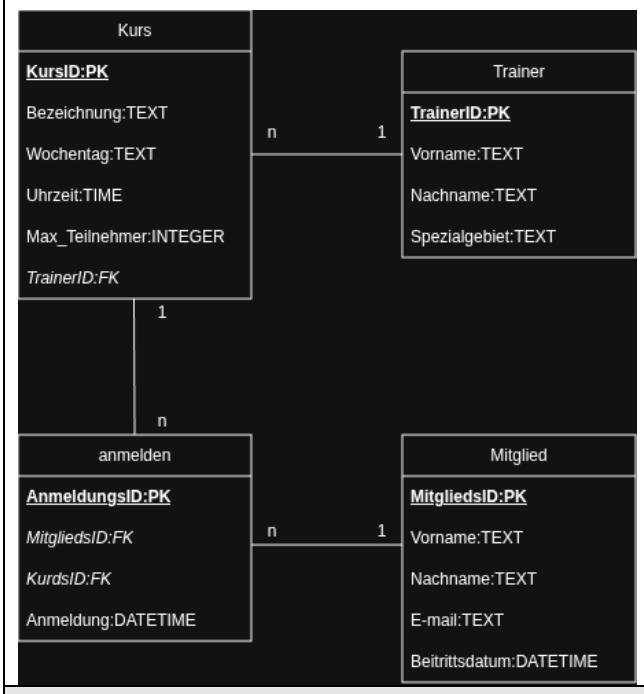
Punkte	**/ 100**				
Note	100-87.5 □ Sehr gut	< 87.5-75 □ Gut	< 75-62.5 □ Befriedigend	< 62.5-50 □ Genügend	< 50 □ Nicht Genügend
Unterschrift					
GitHub-Link	https://github.com/Vlad3750/Fitnessstudio				
Anvil Publish-Link	https://essential-lustrous-blank.anvil.app				

Form	**Punkte**
1. Form der Arbeit (Leserlich, vollständige Abgabe, sinnvolle Dateinamen, Ausführbar, Fehlerbehandlung)	
Szenario: Fitnessstudio-Verwaltung	
Ein lokales Fitnessstudio möchte seine Mitgliederverwaltung und Kursbuchungen digitalisieren. Es gelten folgende Rahmenbedingungen: • Das Fitnessstudio bietet verschiedene Kurse an (z. B. Yoga, Pilates, Spinning, Zumba). Von jedem Kurs werden Bezeichnung, Wochentag, Uhrzeit und maximale Teilnehmeranzahl gespeichert. • Jeder Kurs wird von genau einem Trainer geleitet. Ein Trainer kann mehrere Kurse leiten. Von jedem Trainer werden Vorname, Nachname und Spezialgebiet erfasst. • Mitglieder können sich für beliebig viele Kurse anmelden. Für jeden Kurs können sich mehrere Mitglieder einschreiben. Bei jeder Anmeldung wird das Anmeldedatum gespeichert. Von jedem Mitglied werden Vorname, Nachname, E-Mail-Adresse und Beitrittsdatum erfasst.	
1.1. Aufgabe 1 – Entity-Relationship-Modell (ERM)	**Punkte**
Erstelle auf Basis des Szenarios ein vollständiges ERM: Füge das fertige ERM als Bild in dieses Dokument ein. **(draw.io-Export)**	___ / 15

Name: Neicovcen Vladislav **Datum:** 08.04.2026 **Klasse:** 3AHIF



1.2. Aufgabe 2 – Relationales Modell (RM)	**Punkte**
Leite aus deinem ERM ein vollständiges Relationales Modell ab: Füge das fertige RM als Bild hier ein. **(draw.io-Export)**	___ / 15



1.3. Aufgabe 3 – Python-Skript mit SQLite3 ★ KI-Tools erlaubt	**Punkte**
Schreibe ein Python-Skript Nachname_Vorname_fitnessstudio.py, das: 1. Eine SQLite-Datenbank Nachname_Vorname_fitnessstudio.db erstellt 2. Alle Tabellen aus dem RM anlegt 3. Sinnvolle Beispieldaten einfügt: ≥ 3 Trainer • ≥ 5 Kurse • ≥ 6 Mitglieder • ≥ 8 Anmeldungen	___ / 5

1.4. Aufgabe 4 – Anvil-Implementierung	**Punkte**

Name: Neicovcen Vladislav **Datum:** 08.04.2026 **Klasse:** 3AHIF

Implementiere die Fitnessstudio-Verwaltung als Anvil-Webanwendung (anvil.works).

Pflichtfunktionen:

1. Kursübersicht – Alle Kurse inkl. Trainername

Kurs	Wochentag	Uhrzeit	Trainer	Teilnehmer	Anmelden
Morning Yoga	Montag	08:00	Berger Anna	2/15	Anmelden
Power Lifting	Dienstag	17:00	Huber Markus	2/10	Anmelden
Pilates Basics	Mittwoch	10:00	Mayer Sophie	2/12	Anmelden
HIIT Cardio	Donnerstag	18:30	Schneider Leon	1/20	Anmelden
Abend Yoga	Freitag	19:00	Berger Anna	2/15	Anmelden
Rückenfit	Samstag	09:00	Huber Markus	1/10	Anmelden

2. Neue Anmeldung

Mitglied	Anmelden
Bauer Thomas	Anmelden
Fuchs Stefan	Anmelden
Schwarz Maria	Anmelden
Weiß David	Anmelden
Braun Nina	Anmelden

Bonusfunktionen (optional):

- Mitglieder in einem Kurs anzeigen

___ / 65

1.5. Aufgabe 5 - Abgabe

1. Nachname Vorname in der Kopfzeile eintragen
2. GitHub-Link oben eintragen
3. Anvil Publish-Link oben eintragen
4. Folgende Dateien ins Git-Repo pushen
   a. Nachname_Vorname_ERM.draw.io
   b. Nachname_Vorname_RM.draw.io
   c. Nachname_Vorname_fitnessstudio.py
5. Code hier einfügen

Nachname_Vorname_fitnessstudio.py

```
import sqlite3


conn = sqlite3.connect("Neicovcen_Vladislav_fitnessstudio.db")

conn.execute("PRAGMA foreign_keys = ON")

cursor = conn.cursor()


cursor.executescript("""

CREATE TABLE IF NOT EXISTS Trainer (

TrainerID INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
Vorname TEXT NOT NULL,
Nachname TEXT NOT NULL,
Spezialgebiet TEXT
);

CREATE TABLE IF NOT EXISTS Kurs (
KursID INTEGER PRIMARY KEY AUTOINCREMENT,
Bezeichnung TEXT NOT NULL,
Wochentag TEXT NOT NULL,
Uhrzeit TEXT NOT NULL,
Max_Teilnehmer INTEGER NOT NULL,
TrainerID INTEGER NOT NULL,
FOREIGN KEY (TrainerID) REFERENCES Trainer(TrainerID)
);

CREATE TABLE IF NOT EXISTS Mitglied (
MitgliedsID INTEGER PRIMARY KEY AUTOINCREMENT,
Vorname TEXT NOT NULL,
Nachname TEXT NOT NULL,
E_Mail TEXT NOT NULL UNIQUE,
Beitrittsdatum DATETIME NOT NULL
);

CREATE TABLE IF NOT EXISTS anmelden (
AnmeldungsID INTEGER PRIMARY KEY AUTOINCREMENT,
MitgliedsID INTEGER NOT NULL,
KursID INTEGER NOT NULL,
Anmeldung DATETIME NOT NULL,
FOREIGN KEY (MitgliedsID) REFERENCES Mitglied(MitgliedsID),
FOREIGN KEY (KursID) REFERENCES Kurs(KursID)
);
""")
```

Name: Neicovcen Vladislav **Datum:** 08.04.2026 **Klasse:** 3AHIF

```
cursor.executemany(
"INSERT INTO Trainer (Vorname, Nachname, Spezialgebiet) VALUES (?, ?, ?)",
[
("Anna", "Müller", "Yoga & Meditation"),
("Bernd", "Schmid", "Krafttraining & Bodybuilding"),
("Lisa", "Weber", "Cardio & Ausdauer"),
("Markus", "Huber", "Kampfsport & Selbstverteidigung"),
]
)

cursor.executemany(
"INSERT INTO Kurs (Bezeichnung, Wochentag, Uhrzeit, Max_Teilnehmer, TrainerID) VALUES (?, ?, ?, ?, ?)",
[
("Yoga Grundkurs", "Montag", "09:00", 12, 1),
("Yoga Fortgeschritt", "Donnerstag", "10:00", 8, 1),
("Pump & Burn", "Dienstag", "17:30", 15, 2),
("Rückenfit", "Mittwoch", "18:00", 20, 2),
("Cardio Blast", "Freitag", "07:00", 25, 3),
("Lauftreff", "Samstag", "08:30", 30, 3),
("Kickboxen Basis", "Montag", "19:00", 10, 4),
]
)

cursor.executemany(
"INSERT INTO Mitglied (Vorname, Nachname, E_Mail, Beitrittsdatum) VALUES (?, ?, ?, ?)",
[
("Klaus", "Huber", "k.huber@mail.at", "2023-03-10 09:00:00"),
("Sandra", "Bauer", "s.bauer@mail.at", "2023-07-22 14:00:00"),
("Tobias", "Fischer", "t.fischer@mail.at", "2024-01-05 11:00:00"),
("Maria", "Hofer", "m.hofer@mail.at", "2024-04-18 10:30:00"),
("Felix", "Wagner", "f.wagner@mail.at", "2024-06-30 08:00:00"),
```

```
("Julia", "Berger", "j.berger@mail.at", "2024-09-01 15:00:00"),
("Stefan", "Mayer", "s.mayer@mail.at", "2025-01-12 13:00:00"),
]
)

cursor.executemany(
"INSERT INTO anmelden (MitgliedsID, KursID, Anmeldung) VALUES (?, ?, ?)",
[
(1, 1, "2025-01-05 09:00:00"), # Klaus → Yoga Grundkurs
(1, 5, "2025-01-05 09:10:00"), # Klaus → Cardio Blast
(2, 1, "2025-01-06 10:00:00"), # Sandra → Yoga Grundkurs
(2, 2, "2025-01-06 10:05:00"), # Sandra → Yoga Fortgeschritt
(3, 3, "2025-01-07 14:00:00"), # Tobias → Pump & Burn
(4, 4, "2025-01-08 11:00:00"), # Maria → Rückenfit
(4, 6, "2025-01-08 11:10:00"), # Maria → Lauftreff
(5, 7, "2025-02-01 08:00:00"), # Felix → Kickboxen Basis
(6, 5, "2025-02-03 16:00:00"), # Julia → Cardio Blast
(6, 3, "2025-02-03 16:05:00"), # Julia → Pump & Burn
(7, 4, "2025-03-10 12:00:00"), # Stefan → Rückenfit
(7, 7, "2025-03-10 12:10:00"), # Stefan → Kickboxen Basis
]
)

conn.commit()
conn.close()

print("Datenbank 'fitnessstudio.db' erfolgreich erstellt.")
```

Anvil Frontend Code

Startseite:
from ._anvil_designer import StartseiteTemplate from anvil import * import anvil.tables as tables import anvil.tables.query as q from anvil.tables import app_tables import anvil.server

Name: Neicovcen Vladislav **Datum:** 08.04.2026 **Klasse:** 3AHIF

class Startseite(StartseiteTemplate): def **init**(self, **properties): # Set Form properties and Data Bindings. self.init_components(**properties)

Any code you write here will run before the form opens.

@handle("data_grid_Kurs", "show") def data_grid_Kurs_show(self, **event_args): """This method is called when the data grid is shown on the screen""" return_value = anvil.server.call('get_Kurs') self.repeating_panel_Kurs.items = return_value

Startseite Rowtemplate:

from ._anvil_designer import RowTemplate1Template from anvil import * import anvil.tables as tables import anvil.tables.query as q from anvil.tables import app_tables import anvil.server

class RowTemplate1(RowTemplate1Template): def **init**(self, **properties): # Set Form properties and Data Bindings. self.init_components(**properties)

Any code you write here will run before the form opens.

@handle("Anmelden", "click") def Anmelden_click(self, **event_args): """This method is called when the button is clicked""" open_form('Anmeldungen', kurs=self.item)

Anmeldungen:

from ._anvil_designer import AnmeldungenTemplate from anvil import * import anvil.server import anvil.tables as tables import anvil.tables.query as q from anvil.tables import app_tables

class Anmeldungen(AnmeldungenTemplate): def **init**(self, **properties): # Set Form properties and Data Bindings. self.init_components(**properties)

table = anvil.server.call('app_tables_holen')
angemeldet = {i['mitglied'] for i in table.search(Kurs=Kurs)}

Any code you write here will run before the form opens.

@handle("data_grid_anmelden", "show") def data_grid_anmelden_show(self, **event_args): """This method is called when the data grid is shown on the screen""" return_value = anvil.server.call('get_Mitglied') self.repeating_panel_anmelden.items = return_value

Name: Neicovcen Vladislav **Datum:** 08.04.2026 **Klasse:** 3AHIF

Anmeldungen Rowtemplate:

from ._anvil_designer import RowTemplate2Template from anvil import * import anvil.server import anvil.tables as tables import anvil.tables.query as q from anvil.tables import app_tables

class RowTemplate2(RowTemplate2Template): def **init**(self, **properties): # Set Form properties and Data Bindings. self.init_components(**properties)

Any code you write here will run before the form opens.

@handle("Anmelden", "click") def Anmelden_click(self, **event_args): """This method is called when the button is clicked""" self.remove_from_parent()

Anvil Backend Code

import anvil.tables as tables

import anvil.tables.query as q

from anvil.tables import app_tables

import anvil.files

from anvil.files import data_files

import anvil.server

import sqlite3

This is a server module. It runs on the Anvil server,

rather than in the user's browser.

#

To allow anvil.server.call() to call functions here, we mark

them with @anvil.server.callable.

Here is an example - you can replace it with your own:

#

@anvil.server.callable

def say_hello(name):

print("Hello, " + name + "!")

return 42

#

@anvil.server.callable

def get_Kurs():

Name: Neicovcen Vladislav **Datum:** 08.04.2026 **Klasse:** 3AHIF

```
 with sqlite3.connect(data_files['Neicovcen_Vladislav_fitnessstudio.db']) as conn:
  conn.row_factory = sqlite3.Row
  cur = conn.cursor()
  result = cur.execute("""
   SELECT
     k.Bezeichnung AS Kurs,
     k.Wochentag,
     k.Uhrzeit,
     t.Nachname || ' ' || t.Vorname AS Trainer,
     COUNT(a.AnmeldungsID) || '/' || k.Max_Teilnehmer AS Teilnehmer
   FROM Kurs k
   JOIN Trainer t ON k.TrainerID = t.TrainerID
   LEFT JOIN anmelden a ON k.KursID = a.KursID
   GROUP BY k.KursID
   ORDER BY k.KursID;
  """).fetchall()
  return [dict(row) for row in result]

@anvil.server.callable
def get_Mitglied():
 with sqlite3.connect(data_files['Neicovcen_Vladislav_fitnessstudio.db']) as conn:
  conn.row_factory = sqlite3.Row
  cur = conn.cursor()
  result = cur.execute("""
   SELECT
    Nachname || ' ' || Vorname as Name
   FROM Mitglied
  """).fetchall()
  return [dict(row) for row in result]

@anvil.server.callable
def app_tables_holen():
```

Name: Neicovcen Vladislav **Datum:** 08.04.2026 **Klasse:** 3AHIF

```
table = app_tables.anmelden

return table
```